

# **RARITONE BACKEND PROJECT REPORT**

## **Internship Project Report**

**Submitted By:**

*Nandini Bhimineni*

**Internship Organization:**

*Raritone Private Limited*

**Duration:**

*1 Month*

## DECLARATION

I hereby declare that the project report entitled “**Raritone Backend Project**” submitted in partial fulfillment of the internship requirements is my original work. The information and results presented in this report are based on the work carried out during the internship period.

**Signature:** Nandini Bhimineni

## **ACKNOWLEDGEMENT**

I would like to express my sincere gratitude to the management and technical team of Raritone for providing me with the opportunity to work on this project. Their guidance, support, and valuable insights helped me gain practical knowledge in backend development, database management, API design, and software architecture.

I also thank my faculty members and mentors for their encouragement and support throughout the internship period.

## **ABSTRACT**

Raritone is an innovative virtual fashion platform developed to enhance the online shopping experience through personalization, intelligent recommendations, and virtual fashion technologies. Traditional e-commerce platforms often face challenges such as inaccurate size selection, limited personalization, and the inability for customers to visualize how products will appear before purchase. These limitations frequently result in customer dissatisfaction and increased product return rates. The Raritone platform aims to address these challenges by providing a modern and scalable solution that combines fashion retail with emerging digital technologies.

The primary objective of this project was to design and develop a secure, scalable, and AI-ready backend system capable of supporting various functionalities required for a virtual fashion ecosystem. The backend was built using Node.js and Express.js, with MongoDB serving as the primary database management system. The architecture follows a modular design approach to ensure maintainability, flexibility, and future scalability.

The system includes multiple modules such as user authentication, profile management, product management, wishlist, wardrobe management, shopping cart, order processing, measurement storage, avatar management, and virtual try-on support. Security was implemented using JWT-based authentication, role-based access control, input validation, rate limiting, and secure middleware configurations. Media handling was integrated through Multer and Cloudinary to support efficient image storage and delivery.

In addition to current functionalities, the backend architecture was designed with future AI integration in mind. Features such as AI-powered avatar generation, personalized fashion recommendations, virtual try-on processing, and smart outfit suggestions can be incorporated into the platform with minimal architectural changes.

Overall, the Raritone Backend Project successfully establishes a robust foundation for a next-generation virtual fashion platform, demonstrating modern backend development practices, secure API design, database management, cloud integration, and scalable software architecture.

## **TABLE OF CONTENTS**

1. Introduction
2. Problem Statement
3. Objectives
4. Technology Stack
5. System Architecture
6. Database Design
7. Modules Developed
8. Security Implementation
9. Media Processing Workflow
10. API Development
11. Testing
12. AI Integration Planning
13. Scalability Research
14. Challenges Faced
15. Key Learnings
16. Future Enhancements
17. Conclusion

## **1. INTRODUCTION**

The fashion e-commerce industry faces challenges such as inaccurate size selection, lack of personalization, and uncertainty regarding how products will look on customers. These issues often result in poor customer satisfaction and increased product returns.

Raritone addresses these problems by providing a smart virtual fashion platform capable of supporting virtual try-on technology, personalized recommendations, avatar generation, and user-specific measurements.

The backend acts as the foundation of the platform, managing user data, product information, security, business logic, and future AI integrations.

## **2. PROBLEM STATEMENT**

Traditional online fashion shopping platforms face several limitations:

- Customers cannot visualize outfits before purchasing.
- Size and fit recommendations are often inaccurate.
- Personalized recommendations are limited.
- Managing wardrobe and fashion preferences is difficult.
- Product return rates are high due to fitting issues.

The project aims to solve these challenges by creating a backend architecture capable of supporting AI-powered virtual fashion experiences.

## **3. OBJECTIVES**

The primary objectives of the project are:

- Develop a secure backend API system.
- Implement authentication and authorization.
- Manage products and user profiles.
- Store user measurements.
- Support virtual try-on workflows.
- Manage avatars and personalization data.
- Create scalable architecture for AI integration.
- Ensure data security and performance.
- Build a modular backend structure suitable for future expansion.

## **4. TECHNOLOGY STACK**

### **Backend Technologies**

- Node.js
- Express.js

### **Database**

- MongoDB
- Mongoose ODM

### **Authentication**

- JWT Authentication
- Google OAuth

### **Media Handling**

- Multer
- Cloudinary

### **Security**

- Helmet
- Express Rate Limiting
- CORS

### **Logging**

- Morgan

### **Validation**

- Joi Validators

### **Future Technologies Researched**

- Redis
- Docker
- AWS
- Load Balancers
- Queue Systems
- Microservices

## 5. SYSTEM ARCHITECTURE

The backend follows a modular layered architecture.

Frontend / Mobile Application



Backend API Layer



Authentication & Security Layer



Business Logic Layer



Database Layer (MongoDB)



Cloudinary Storage



AI Services (Future Integration)

### Advantages

- Easy maintenance
- Better scalability
- Modular development
- Improved security
- AI integration readiness

## 6. DATABASE DESIGN

MongoDB was used as the primary database.

### Collections Implemented

1. User
2. Profile
3. Product
4. Wishlist



5. Wardrobe
6. Cart
7. Order
8. Measurement
9. Avatar
10. ProductMapping
11. TryOn

### **Relationships**

User → Profile

User → Wishlist

User → Wardrobe

User → Cart

User → Orders

User → Measurements

User → Avatars

Wishlist → Product

Cart → Product

Order → Product

ProductMapping → Product

Mongoose Populate functionality was implemented to retrieve related information efficiently.

## **7. MODULES DEVELOPED**

### **Authentication Module**

#### **Features**

- User Registration
- User Login
- Logout
- Password Reset

- Google OAuth Login
- JWT Authentication
- Refresh Tokens

#### **Benefits**

- Secure user management
- Protected API access
- Role-based authorization

#### **Product Module**

##### **Features**

- Product Creation
- Product Updates
- Product Deletion
- Product Search
- Product Filtering

##### **Benefits**

- Efficient catalog management
- Product organization
- Administrative control

#### **Profile Module**

##### **Features**

- User Profile Management
- Avatar Upload Support
- User Preferences

##### **Benefits**

- Personalization
- Better user experience

#### **Wishlist Module**

##### **Features**

- Save favorite products
- View wishlist items
- Remove wishlist products

#### **Benefits**

- Personalized shopping experience

#### **Wardrobe Module**

##### **Features**

- Store clothing items
- Manage wardrobe collection
- Organize fashion preferences

##### **Benefits**

- Personalized styling support

#### **Cart Module**

##### **Features**

- Add products to cart
- Remove products
- View cart details

##### **Benefits**

- Simplified purchase workflow

#### **Order Module**

##### **Features**

- Order creation
- Order management
- Order history

##### **Benefits**

- Complete e-commerce functionality

#### **Measurement Module**

##### **Features**

- User body measurements
- Personalized size support

### **Benefits**

- Better fitting recommendations
- Reduced return rates

### **Avatar Module**

#### **Features**

- Avatar information storage
- Body type support
- Skin tone support
- Gender support

#### **Benefits**

- Future AI avatar generation
- Virtual try-on compatibility

### **Product Mapping Module**

#### **Features**

- Product relationships
- Recommendation preparation
- AI compatibility support

#### **Benefits**

- Personalized product suggestions
- Smart recommendations

### **Try-On Module**

#### **Features**

- Virtual try-on request handling
- User image support
- AI-ready architecture

#### **Benefits**

- Better visualization before purchase

## **8. SECURITY IMPLEMENTATION**

The backend includes multiple security layers.

### **JWT Authentication**

Protects sensitive APIs.

### **Role-Based Access Control (RBAC)**

Roles:

- USER
- ADMIN
- SUPER\_ADMIN

### **Helmet**

Provides security headers.

### **Rate Limiting**

Prevents abuse and DDoS-style attacks.

### **Validation**

Ensures clean and valid input data.

### **Error Handling Middleware**

Centralized error management.

## **9. MEDIA PROCESSING WORKFLOW**

User Upload

↓

Multer Upload

↓

Sharp Compression

↓

Cloudinary Upload

↓

CDN Delivery



Response

### **Benefits**

- Faster loading
- Reduced storage
- Optimized images

## **10. API DEVELOPMENT**

RESTful APIs were developed for:

- Authentication
- Products
- Profiles
- Wishlist
- Wardrobe
- Cart
- Orders
- Measurements
- Avatar
- Try-On

All APIs follow structured request-response patterns and support authentication where required.

## **11. TESTING**

The system was tested using Postman.

### **Testing Areas**

- Authentication
- CRUD Operations
- Validation
- Protected Routes
- Error Handling

- Database Integration

## **Results**

All core modules operated successfully without critical issues.

## **12. AI INTEGRATION PLANNING**

The project was designed to support future AI services.

### **Planned Features**

- AI Avatar Generation
- Recommendation Engine
- Smart Outfit Suggestions
- Virtual Try-On Processing
- Personalized Fashion Analytics

## **13. SCALABILITY RESEARCH**

Several scalability solutions were researched.

### **Redis**

For caching frequently accessed data.

### **Queue Systems**

For background processing.

### **Load Balancing**

For handling increased traffic.

### **Docker**

For containerized deployment.

### **Microservices**

For future architecture expansion.

### **MongoDB Atlas Scaling**

For database growth management.

## **14. CHALLENGES FACED**

### **Challenge 1: Designing a Scalable Architecture**

**Solution:** Implemented a modular backend structure.

### **Challenge 2: Managing Relationships Between Multiple Modules**

**Solution:** Used MongoDB references and Populate functionality.

### **Challenge 3: Security Implementation**

**Solution:** Implemented JWT, RBAC, Helmet, and Rate Limiting.

### **Challenge 4: Planning Future AI Workflows**

**Solution:** Created AI-ready database schemas and architecture.

## **15. KEY LEARNINGS**

Throughout the internship, significant knowledge was gained in:

- Node.js Development
- Express.js Framework
- MongoDB Database Design
- JWT Authentication
- Google OAuth
- REST API Development
- Backend Architecture
- Security Best Practices
- Cloud Storage Concepts
- Documentation Writing
- Git & GitHub Collaboration
- Team-Based Software Development

## **16. FUTURE ENHANCEMENTS**

Planned improvements include:

- Full AI Avatar Generation
- Recommendation Engine
- Redis Integration
- Queue Processing
- Docker Deployment
- AWS Infrastructure



- Real-Time Notifications
- Socket.IO Integration
- Monitoring Systems
- Advanced Analytics

## **17. CONCLUSION**

The Raritone Backend Project successfully established a secure, scalable, and AI-ready backend foundation for a virtual fashion platform. The project implemented essential e-commerce and personalization features while preparing the system for future AI-driven enhancements.

Through this project, valuable experience was gained in backend development, API design, database management, security implementation, testing, cloud concepts, and software architecture. The internship provided practical exposure to real-world software development practices and strengthened technical skills required for building scalable modern applications.